Orderflow Engine: arquitectura de un servicio multipaís de cotización y confirmación de pedidos

Jesús Ariel González Bonilla

2026-10-17

Pregunta que responde este documento: ¿qué artefactos componen Orderflow Engine, y qué hace cada uno y por qué?

Orderflow Engine — Prueba técnica de Tech Lead

Resumen

Orderflow Engine cotiza y confirma los pedidos de tenderos a distribuidores de bebidas en un marketplace B2B multi-tenant y multipaís. Este informe describe su arquitectura e inventaría cada artefacto de los seis repositorios que lo entregan: un servicio en Java 21 y Spring Boot 4.1.1, un cliente web en Angular 22, la infraestructura con Docker Compose, la documentación, las herramientas de verificación y una carpeta reservada para una futura app móvil. El servicio es un monolito modular hexagonal cuyo dominio de precios, puro, aplica obsequios, combos, descuentos por escala marginales e impuestos por línea en un orden fijo y justificado, con dinero en BigDecimal redondeado solo por línea a la escala del país. Las cotizaciones son selladas y de solo inserción, y el pedido las referencia sin guardar montos, así que el total confirmado es el cotizado por construcción. La confirmación corre en una transacción de PostgreSQL 17 que bloquea la clave de idempotencia, la cotización y el cupo, en ese orden, y responde 201 solo tras el commit. Los efectos hacia push y el ERP salen por un outbox transaccional, y los consumidores los deduplican por el id del evento. Cubre además el modelo de datos, la internacionalización, la entrega y la evidencia verificada: 333 pruebas unitarias y 308 de integración, cobertura total del dominio de precios, 185 de 185 mutaciones detectadas, 209 specs web y un flujo de punta a punta en dos idiomas. Cierra con los riesgos, sus mitigaciones y lo que se dejó fuera.

Palabras clave: motor de precios, idempotencia, outbox transaccional, arquitectura hexagonal, aritmética exacta de dinero, multi-tenant, PostgreSQL, Spring Boot, Angular

Contexto y objetivos de calidad

El enunciado

El enunciado, que se conserva sin cambios en orderflow-engine-docs/assignment/, describe un marketplace B2B donde los tenderos compran a distribuidores de bebidas. La plataforma opera en varios países (Colombia, Ecuador, Guatemala y Perú, entre otros) y para varias marcas comerciales, sobre una misma plataforma multi-tenant. Un pedido pasa por catálogo, carrito, checkout (la cotización), confirmación e historial. Cinco palancas comerciales actúan sobre el precio y pueden coexistir en un mismo pedido: el descuento por escala por SKU o familia, el combo, el obsequio, la tasa de impuesto por SKU y el cupo de crédito del cliente.

La prueba tiene tres partes. A1 pide el motor de cálculo: un desglose por línea y totales, un orden de palancas definido y justificado, y la verificación del cupo. A2 pide una confirmación de pedidos en la que se vuelva imposible un defecto visto en tres releases: al tendero se le devolvía un número de pedido y después el pedido no aparecía en su

historial, a veces porque nunca había quedado registrado. A3 pide un único registro de decisión de arquitectura de una página como máximo. La entrega es un repositorio Git con un comando único de pruebas, un README que explique cómo correr el servicio, el orden de las palancas, la política de redondeo y el modelo de consistencia, el ADR y un historial de commits legible. Un front opcional, limitado a una hora, es una pantalla de resumen del pedido que se juzga por tres defectos pasados: textos cortados, controles tapados por la barra del sistema o por el teclado, e imágenes de un grid que escalan con criterios distintos.

Objetivos de calidad

El enunciado presenta sus requisitos no funcionales como el corazón del ejercicio. La especificación (orderflow-engine-docs/spec/SPEC.md §0) los convierte en cuatro garantías, y la constitución (spec/constitution.md) agrega quince principios, de P-01 a P-15, cada uno con la prueba o verificación que lo comprueba. La Tabla 1 resume los objetivos de calidad, el mecanismo que cumple cada uno y la evidencia que lo demuestra.

Tabla 1. Objetivos de calidad, mecanismos y evidencia

Objetivo	Mecanismo	Verificado por
G-1: el total cotizado es el total confirmado, aunque cambien precios o palancas entre un momento y otro	Una cotización sellada de solo inserción. El pedido la referencia y no guarda montos	QuoteConfirmRegressionIT (golden test 6)
G-2: nada de punto flotante para dinero; escala, regla y puntos de redondeo explícitos	Un objeto de valor Money sobre BigDecimal, con escala y modo tomados de los datos del país, y redondeo por línea	DomainPurityArchTest, MoneyTest, RoundingPolicyTest
G-3: un 201 significa que el pedido existe y se puede consultar; los reintentos nunca duplican pedidos ni consumen el cupo dos veces	Una transacción ACID para pedido, cupo, clave de idempotencia y outbox; la respuesta tras el commit	ConfirmOrderCommitOrderingIT, OrderReadYourWritesIT, ConfirmOrderConcurrencyIT
G-4: los efectos hacia push y el ERP nunca se pierden ni se aplican dos veces	Outbox transaccional, relay con lease y reintentos, deduplicación por event_id	OutboxWriteIT, OutboxRelayIT
Multipaís: ninguna moneda, escala ni tasa en el código	Datos de país, tenant, precios y palancas en tablas	GoldenPricingTest (caso multipaís), las semillas de Liquibase
Front: sin textos cortados, sin controles tapados, imágenes consistentes	Un componente es dueño de cada defecto	order-summary.spec.ts, screen-rules.spec.ts

Límite del alcance

SPEC §1 fija el límite. Dentro del alcance están las operaciones de cotización, pedido, historial y lista de productos, el motor de precios con su verificación de cupo, el relay del outbox y la limpieza por retención, la internacionalización en español (por defecto) e inglés, el cliente Angular y la ejecución local con Docker. Fuera del alcance, cada uno con su razón, quedan la autenticación (tenant y cliente llegan en cabeceras), la administración de catálogo y palancas (tiendas, productos y palancas se siembran con changesets de Liquibase), las integraciones reales con push y el ERP (hay fakes detrás de puertos), el carrito en el servidor, el cobro, la cancelación y la liberación de cupo, la revocación de cotizaciones, el despliegue, la infraestructura en la nube y un archivo OpenAPI.

El front tiene una única desviación registrada. El 2026-10-15 el dueño del producto amplió la pantalla de resumen de una hora al flujo completo: carrito, cotización, confirmación, pedido y Mis pedidos, con un selector de tienda. El entregable de la API no depende de él ni lo espera, y el tiempo medido queda registrado en tres partes en el README de la API, en "No incluido": la pantalla del enunciado tomó 68 minutos; su contenedor, el stack completo y el paso web del comando de pruebas del espacio de trabajo, 55 minutos; y la ampliación, 3 horas y 1 minuto.

Fuentes y convenciones de este informe

Cada afirmación sobre el sistema se verificó leyendo los repositorios: código fuente, changelogs, archivos de Docker y Compose, scripts, READMEs, el conjunto de la especificación y el handoff. Los artefactos del proyecto se citan por su ruta relativa al espacio de trabajo y por sección o código (por ejemplo, SPEC §7.1, DEC-05 o AC-26), porque son el objeto del informe y no literatura. Las obras externas se citan con el estilo autor-fecha de la American Psychological Association (2020). Las figuras son los diagramas generados de orderflow-engine-docs/diagrams/, numerados por primera aparición, así que la Figura 1 es 01-runtime-topology y la Figura 9 es 09-outbox-lifecycle; sus rótulos internos están en inglés, como toda la documentación técnica. Las cifras verificadas vienen de la tabla de estado de orderflow-engine-docs/handoff/HANDOFF.md §3, el único lugar donde vive el estado verificado.

Visión general del sistema

Topología de ejecución

En ejecución, el sistema tiene un servicio desplegable, una base de datos y un cliente web. Como muestra la Figura 1, el navegador habla solo con el contenedor web, cuyo nginx sirve el cliente Angular compilado y reenvía /api/ al servicio. El servicio contiene el adaptador web, los casos de uso, el motor de precios puro, el adaptador de persistencia y el relay del outbox. PostgreSQL es la única fuente de verdad. Push y el ERP solo se alcanzan desde el relay, después del commit del pedido (principios P-04 y P-05).

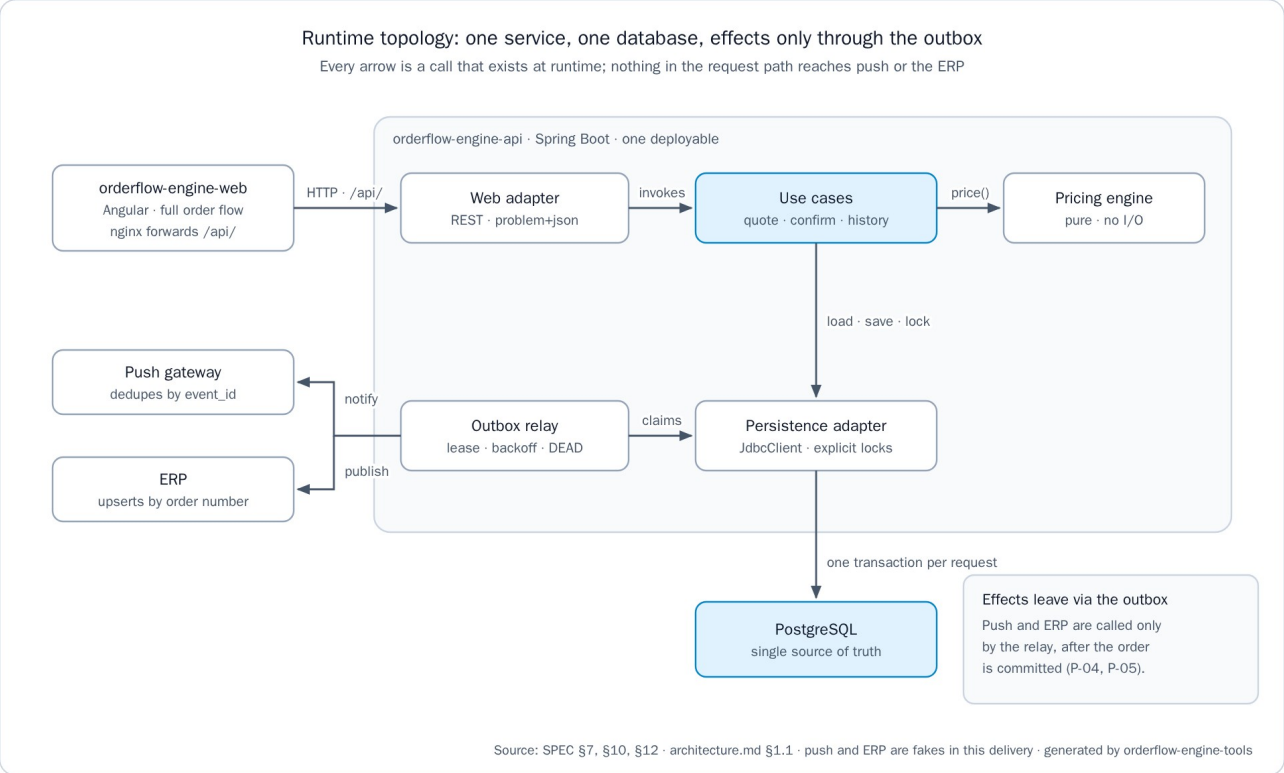


Figura 1. Topología de ejecución: un servicio, una base de datos y efectos solo a través del outbox

La Tabla 2 lista los elementos de ejecución del stack completo que levanta orderflow-engine-infra/compose.yaml.

Tabla 2. Elementos de ejecución del stack completo

Elemento	Tecnología	Puerto	Responsabilidad
Cliente web	Angular 22.2.3, servido por nginx-unprivileged 1.30.6	8081 en el host, 8080 dentro del contenedor	Sirve el cliente y reenvía /api/ al servicio
Servicio	Java 21, Spring Boot 4.1.1	8080	Cotiza carritos, sella cotizaciones, confirma pedidos, sirve el historial y la lista de productos, entrega los eventos del outbox y ejecuta la limpieza
Base de datos	PostgreSQL 17	No se publica en el host	Catálogo, palancas, cupo, cotizaciones, pedidos, claves de idempotencia y outbox
Gateways de push y ERP	Adaptadores fake dentro del servicio	—	Reciben el pedido confirmado, deduplican por event_id y se les puede ordenar que fallen

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 4

Repositorios del espacio de trabajo

El espacio de trabajo son seis carpetas hermanas, cada una un repositorio de GitHub, privado durante el trabajo y público en la entrega, con su propio README en inglés y español y una licencia MIT. Como muestra la Figura 2, la especificación de orderflow-engine-docs gobierna a todos, orderflow-engine-tools genera las figuras y los entregables que se guardan en el repositorio de documentación, y orderflow-engine-infra construye y prueba el servicio y el cliente web. La decisión DEC-15 eligió repositorios separados en lugar de un monorepo porque el servicio, la pantalla, la infraestructura y la documentación se publican, tienen dueño y evolucionan por separado, mientras orderflow-engine-api sigue siendo, por sí solo, el entregable.

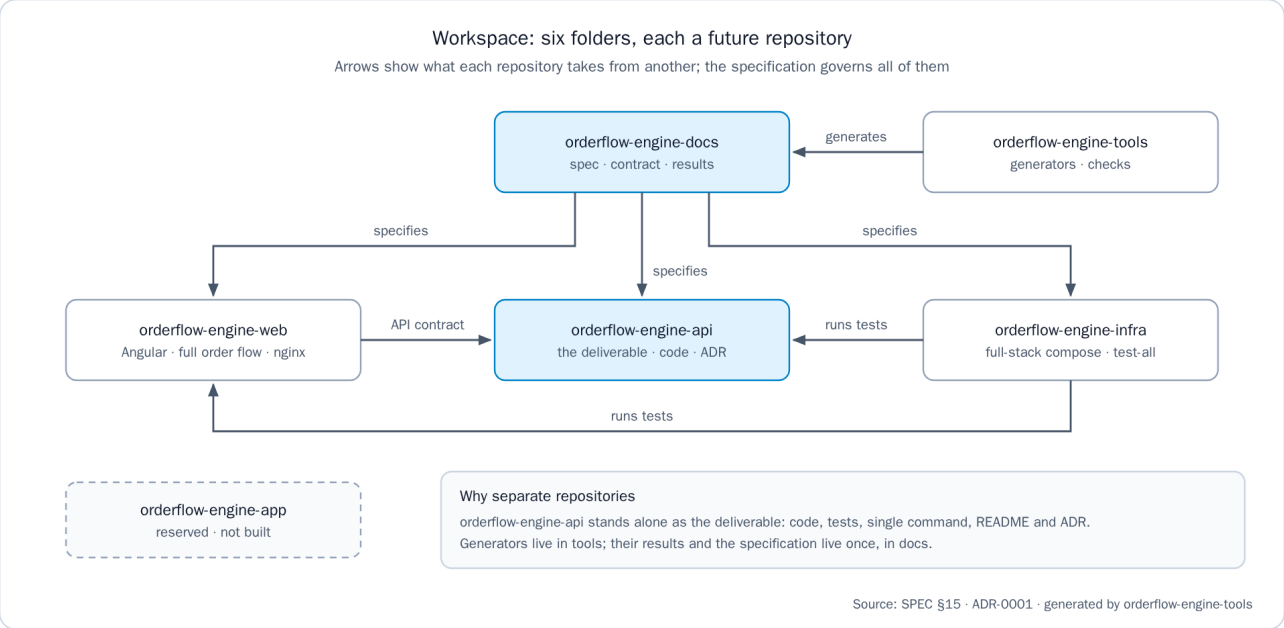


Figura 2. Espacio de trabajo: seis carpetas, cada una un futuro repositorio

La Tabla 3 resume el papel de cada repositorio y sus ramas permanentes.

Tabla 3. Repositorios del espacio de trabajo

Repositorio	Papel	Ramas permanentes
orderflow-engine-api	El entregable: servicio, pruebas, comando único, README y ADR	main, dev
orderflow-engine-web	El cliente Angular: el resumen del pedido y el flujo alrededor	main, dev
orderflow-engine-infra	Compose del stack completo, comando de pruebas del espacio de trabajo, suite de punta a punta	main, dev
orderflow-engine-docs	Especificación, gobierno, handoff, figuras, entregables generados	Solo main
orderflow-engine-tools	Generadores y verificaciones de documentos y repositorios	main, dev
orderflow-engine-app	Reservado para una futura app móvil; no se construye nada	main, dev

Inventario de artefactos

Esta sección inventaría los artefactos de cada repositorio. Las tablas nombran cada artefacto por su ubicación y dicen qué contiene y por qué existe.

El repositorio del servicio

orderflow-engine-api contiene todo el código y las pruebas que evalúa el enunciado. Sus fuentes Java viven en src/main/java/com/orderflow/engine, y la Tabla 4 las lista junto con la compilación, los recursos, las pruebas y los archivos de contenedor.

Tabla 4. Artefactos de orderflow-engine-api

Ubicación	Contenido y propósito
pom.xml, mvnw, mvnw.cmd, .mvn/	Compilación con Maven Wrapper sobre el parent de Spring Boot 4.1.1 para Java 21. ./mvnw verify ejecuta todas las pruebas y el umbral de cobertura de JaCoCo 0.8.15; el perfil opcional mutation ejecuta PIT 1.30.1
pricing/domain	Money, CurrencyPolicy, Cart, CartLine, CartNormalizer, PricingSnapshot, PricingEngine, Breakdown, LineBreakdown, Totals, Discount, ComboAllocation, Gift
pricing/domain/lever	Los pasos y reglas de las palancas: GiftStep, GiftRule, GiftTaxPolicy, ComboStep, ComboRule, LargestRemainderAllocator, TierDiscountStep, TierRule
quoting	CreateQuoteUseCase, GetQuoteUseCase, ListProductsUseCase, el QuoteStatus derivado, y los puertos SnapshotProvider, QuoteRepository, CreditAdvisor, ProductCatalog
ordering	ConfirmOrderUseCase, GetOrderUseCase, ListOrdersUseCase, y los puertos OrderRepository, CreditLedger, IdempotencyStore, OutboxWriter
outbox	OutboxRelay, BackoffPolicy, HousekeepingJob, y los puertos PushNotificationPort, ErpPublisherPort, OutboxRepository, HousekeepingStore
adapters/in/web	QuoteController, OrderController, ProductController, los records de petición y respuesta, ProblemDetailsAdvice, ErrorCode, CorrelationIdFilter, LocaleConfig, RequestHash
adapters/out/persistence	Repositorios con JdbcClient, SQL explícito y bloqueos, LockWaits, MoneyColumns, UuidV7Generator
adapters/out/push, adapters/out/erp	FakePushGateway y FakeErpGateway, que deduplican por event_id y a los que se les puede ordenar que fallen
config	La raíz de composición: QuotingConfig, OrderingConfig, OutboxConfig (relay y gauges), HousekeepingConfig, TimeConfig (el Clock inyectado) y propiedades tipadas

application.yml, .env.example	Valores por defecto de cada clave app.*. La conexión a la base de datos llega solo por variables de entorno, sin secretos por defecto
db/changelog	Un changelog maestro que incluye ocho changelogs (33 changesets) en orden: cinco crean el esquema, uno siembra Colombia y Ecuador, uno renombra las semillas en español y uno agrega Guatemala y Perú
i18n	messages.properties (base en español), messages_es.properties y messages_en.properties
src/test	Pruebas unitarias, golden, de contrato, de integración y de arquitectura, y once carpetas de fixtures golden en golden/
Dockerfile, compose.yaml	Una imagen de dos etapas (compilación en JDK 21 con el wrapper, ejecución en JRE 21 con un usuario sin privilegios, una capa por cada capa de Spring Boot) y el servicio con su propio PostgreSQL 17
README.md, README.es.md, docs/adr/	Cómo ejecutarlo, orden de las palancas, política de redondeo, modelo de consistencia, golden tests, ADR, no incluido, operación y repositorios; el ADR-0001 en inglés y español

El repositorio del cliente web

orderflow-engine-web es una aplicación Angular independiente (DEC-16) que consume la API por rutas relativas /api/. La Tabla 5 lista sus artefactos.

Tabla 5. Artefactos de orderflow-engine-web

Ubicación	**Contenido y propósito**
package.json, angular.json	Angular 22.2.3, ngx-translate 18.0.0, Vitest 5.0.5 con su proveedor de navegador Playwright, y Playwright 1.64.1. El builder de pruebas unitarias ejecuta las specs en Chromium headless
src/app/app.routes.ts	Las cuatro pantallas: /cart (también /), /quotes/:quoteId, /orders/:orderId y /orders. Cualquier otra ruta abre el carrito
src/app/pages	Las pantallas cart, order-summary, order y history
src/app/breakdown	Las líneas y los totales del desglose, compartidos por el resumen del pedido y el pedido
src/app/ui	ui-text, ui-action-bar, ui-product-image, ui-problem-notice y ui-notice
src/app/shell	El encabezado: navegación, el selector ES/EN y el selector de tienda
src/app/core	Clientes de la API para cotizaciones, pedidos y productos, el interceptor de cabeceras, el pipe money, el almacén del carrito, las claves de idempotencia, el contexto de tienda, el idioma, el manejo de problemas y el almacenamiento del

	navegador
src/testing	Fixtures con nombres muy largos, el fixture del resumen con la forma de E-02, verificaciones de pantalla y un arnés de pruebas
public/i18n	Los diccionarios es.json y en.json
public/config/stores.json	Las ocho tiendas sembradas del selector de tienda, dos por tenant
public/products	Una imagen SVG por cada SKU sembrado y un placeholder
scripts/no-literal-text.mjs	Corre primero en npm test y falla cuando una plantilla tiene texto literal
nginx/default.conf.template	El servidor nginx: /api/ se reenvía a API_UPSTREAM y toda otra ruta cae en index.html
Dockerfile, compose.yaml, .env.example, proxy.conf.mjs	Una imagen de tres etapas (specs sobre la imagen de Playwright, compilación en Node 24.21.0, servicio en nginx-unprivileged 1.30.6), el cliente solo contra una API en el host, y el proxy del servidor de desarrollo
README.md, README.es.md	Alcance, cómo ejecutarlo, prevención a nivel de componente y qué prueba cada spec

El repositorio de infraestructura

orderflow-engine-infra es el único repositorio que referencia a sus hermanos, porque los construye y los ejecuta juntos. La Tabla 6 lista sus artefactos.

Tabla 6. Artefactos de orderflow-engine-infra

Ubicación	**Contenido y propósito**
compose.yaml	El stack completo: db (postgres:17 con health check y un volumen con nombre), api (construido desde ../orderflow-engine-api) y web (construido desde ../orderflow-engine-web, con API_UPSTREAM). Todos los servicios tienen restart: unless-stopped
.env.example	Cada clave con un comentario y su valor por defecto: base de datos, puerto e idioma de la API, outbox, retención y puerto web
scripts/test-all.sh, scripts/test-all.ps1	El comando de pruebas del espacio de trabajo en cinco pasos, para bash y PowerShell
scripts/e2e.sh, scripts/e2e.ps1	El paso de punta a punta: verificación de versión, imagen del ejecutor, un stack en sus propios puertos, la suite y un desmontaje incondicional
e2e/	La suite de Playwright (tests/full-flow.spec.ts), su configuración, el Dockerfile de su ejecutor sobre la imagen de Playwright 1.64.1, y check-playwright-version.mjs
README.md, README.es.md	Disposición de los clones, cómo ejecutarlo, el comando de pruebas y el flujo de punta a punta

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 8

El repositorio de herramientas

orderflow-engine-tools mantiene los generadores y las verificaciones aparte de los documentos que producen (DEC-18). La Tabla 7 los lista.

Tabla 7. Artefactos de orderflow-engine-tools

Ubicación	Contenido y propósito
docs-check/check-docs.mjs	La compuerta de documentación: ubicaciones permitidas, mapa de documentos, pares de idioma con el mismo número de encabezados, enlaces, front matter, encabezados obligatorios, extensión del ADR, paridad de claves i18n, gráfico de avance, líneas de commit y fuentes de los entregables
integrity/check-integrity.mjs	Archivos de ignore, .gitattributes, .env.example donde un repositorio corre con Docker, archivos Docker que no salen de su repositorio, y ningún secreto
progress/progress.mjs	El gráfico de avance de tareas, generado desde la tabla de tareas del handoff
diagrams/generate_diagrams.py	Las nueve figuras SVG, verificadas contra textos desbordados, rótulos superpuestos, aristas que se cruzan y aristas que atraviesan cajas
render/render_document.py, render/build_deliverables.py	Markdown a Word con la plantilla de impresión (A4, Arial 12, texto negro, márgenes de 2 cm) y luego PDF con LibreOffice, con el SHA-256 de cada fuente en manifest.json
consistency/	Verificaciones de referencias cruzadas, conteos declarados, fugas de idioma y paridad entre inglés y español
Dockerfile, requirements.txt	Una imagen con Python 3.11, Node y los paquetes de Python; las herramientas corren desde el espacio de trabajo montado

El repositorio de documentación

orderflow-engine-docs es dueño de la especificación y de cada resultado generado. Su raíz contiene solo los READMEs y la licencia, y un documento que no está en su mapa de documentos no existe (regla de documentación DR-2). La Tabla 8 lista sus carpetas.

Tabla 8. Artefactos de orderflow-engine-docs

Ubicación	Contenido y propósito
assignment/	El enunciado original, que nunca se edita
spec/	Constitución, SPEC, plan técnico, TASKS, contrato de la API, modelo de datos y arquitectura
governance/	Regla de oro, ramas y entornos, reglas de pull requests, convenciones de commits, reglas de documentación, integridad de repositorios, la configuración de verificación docs-governance.json, y las plantillas de ADR y de documento

handoff/	HANDOFF.md, con el estado verificado, los comandos para retomar, las decisiones abiertas y las trampas, y progress.svg
diagrams/	Las nueve figuras SVG generadas
deliverables/	El ADR-0001 en Word y PDF, en inglés y español, y manifest.json
delivery/	Los documentos de entrega en inglés y español, entre ellos este informe, con las capturas de pantalla y las imágenes de las figuras
README.md, README.es.md	El glosario de carpetas, el mapa de documentos y los códigos que se citan entre documentos

El repositorio reservado de la app

orderflow-engine-app contiene solo sus READMEs y su licencia. Está reservado para un futuro cliente móvil, de modo que una app móvil tiene dónde vivir sin que se construya nada en esta entrega.

Arquitectura del servicio

Estructura hexagonal

El servicio es un monolito modular hexagonal (DEC-03), en el sentido de puertos y adaptadores que describe Cockburn (2005): el núcleo de la aplicación está rodeado de puertos, y la tecnología vive en adaptadores a ambos lados. Como muestra la Figura 3, el lado que dirige tiene los controladores REST y los jobs programados, el núcleo tiene los casos de uso de quoting, ordering y outbox alrededor del pricing.domain puro, y el lado dirigido tiene el adaptador JDBC y los gateways fake de push y ERP. Se descartaron los microservicios porque una sola transacción debe abarcar el pedido, el cupo, la clave de idempotencia y el outbox, y separarlos obligaría a una saga sin ningún beneficio.

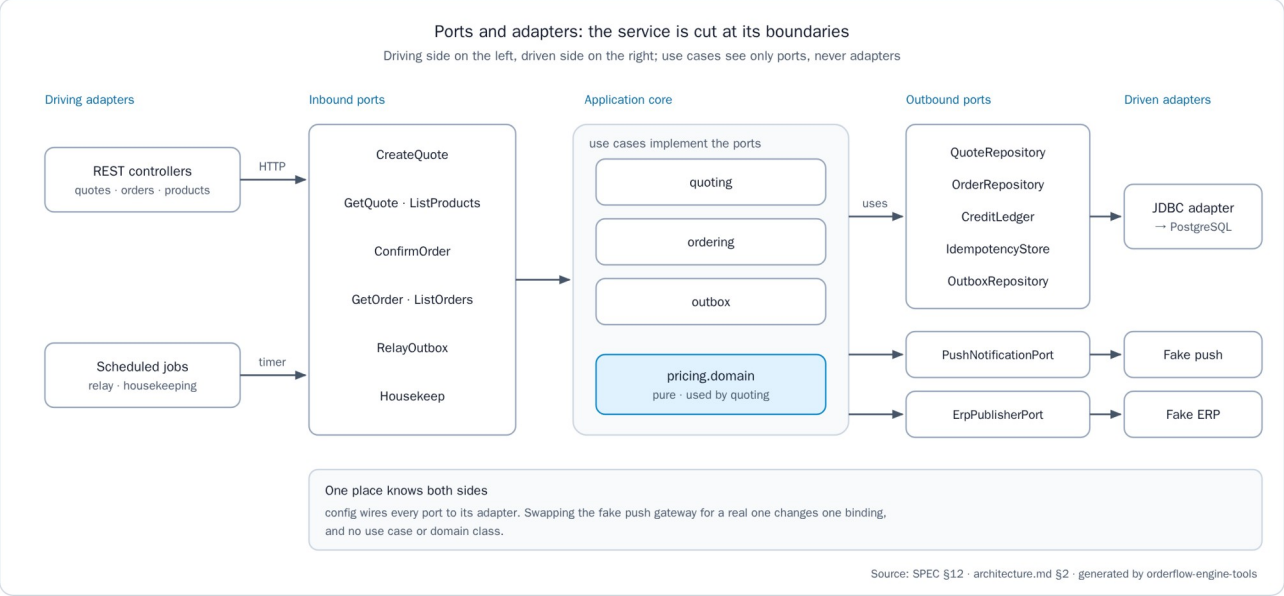


Figura 3. Puertos y adaptadores: el servicio se corta en sus fronteras

La Tabla 9 lista los módulos bajo el paquete base com.orderflow.engine y de qué puede depender cada uno. El paquete config es la raíz de composición: cambiar el gateway fake de push por uno real cambia un solo enlace y ningún caso de uso ni clase del dominio.

Tabla 9. Módulos del servicio

Módulo	Tipo	Depende de
pricing.domain	Dominio, puro	Solo el JDK
quoting	Aplicación	pricing.domain
ordering	Aplicación	Puertos de quoting, pricing.domain
outbox	Aplicación	Sus propios puertos
adapters.in.web	Adaptador que dirige	Puertos de la aplicación
adapters.out.persistence	Adaptador dirigido	Puertos de la aplicación
adapters.out.push, adapters.out.erp	Adaptadores dirigidos	Puertos del outbox
config	Raíz de composición	Todos los módulos

El dominio sigue la separación que recomienda Evans (2003) entre el modelo del dominio y la infraestructura que lo rodea: Money y CurrencyPolicy son objetos de valor inmutables, y el motor es una función pura price(cart, snapshot) que no lee el reloj, no guarda estado y no hace entrada ni salida. El tiempo es un Clock inyectado que las pruebas fijan, así que los golden tests y el vencimiento de las cotizaciones son deterministas.

La regla de dependencias y cómo se hace cumplir

Las dependencias del código apuntan hacia adentro. Como muestra la Figura 4, los adaptadores dependen de la aplicación, la aplicación depende del dominio, el dominio importa solo el JDK, y config es el único módulo que conoce todas las capas. Una dependencia en sentido contrario rompe la compilación, porque las reglas de ArchUnit (TNG Technology Consulting, s. f.) corren con las demás pruebas en ./mvnw verify.

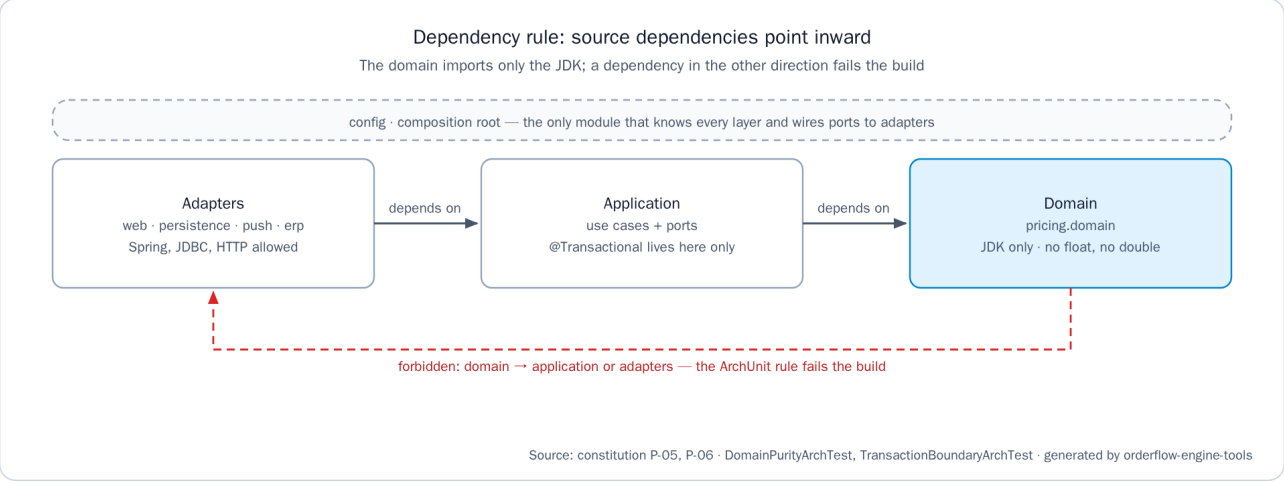


Figura 4. Regla de dependencias: las dependencias del código apuntan hacia adentro

La Tabla 10 lista las diez reglas y la razón de cada una. Las cuatro primeras pertenecen a DomainPurityArchTest y las otras seis a TransactionBoundaryArchTest.

Tabla 10. Reglas de arquitectura que hace cumplir ArchUnit

Ámbito	Regla	Razón
Dominio	El dominio de precios depende solo	El motor es puro y

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 11

	del JDK y de sí mismo	determinista (P-06)
Dominio	El dominio de precios no usa ningún tipo de java.sql ni de javax.sql	El dominio nunca llega a la base de datos
Dominio	Ningún campo, parámetro, tipo de retorno, llamada ni dependencia del dominio usa float, double, Float o Double	El dinero nunca es punto flotante binario (AC-08)
Dominio	El dominio nunca usa java.util.Currency	La escala sale de los datos del país; el JDK reporta el COP con escala 2
Transacciones	Los métodos @Transactional se declaran solo en clases de aplicación llamadas *UseCase	Un controlador respondería antes del commit, y un adaptador partiría una confirmación en varias
Transacciones	Una clase anotada entera con @Transactional es un caso de uso	La frontera transaccional es el caso de uso
Transacciones	Un método transaccional se llama solo desde otras clases	Una llamada desde su propia clase salta el proxy y corre sin transacción
Transacciones	Una clase con un método transaccional nunca depende de un cliente HTTP ni de los gateways de push o ERP	Ninguna llamada remota dentro de una transacción de base de datos (P-04, P-05)
Transacciones	Los paquetes de aplicación nunca dependen de adaptadores ni de config	Los casos de uso ven puertos, nunca adaptadores
Transacciones	ordering nunca usa PricingEngine ni CartNormalizer, y nunca carga un snapshot de precios	La confirmación nunca vuelve a tarifar, así que el total confirmado es el cotizado

La regla del proxy existe porque Spring aplica @Transactional mediante un proxy, de modo que una llamada desde la misma clase corre en silencio sin transacción (Spring, s. f.). SPEC §7.6 registra ese salto como una causa raíz plausible del defecto reportado.

Superficie de la API y modelo de errores

El servicio expone siete operaciones, especificadas como fichas E-01 a E-07 en spec/api-contract.md. Toda operación de negocio exige las cabeceras X-Tenant-Id y X-Customer-Id, Accept-Language es opcional, y el dinero es un string JSON con la escala de la moneda, nunca un número JSON. La Tabla 11 lista las operaciones.

Tabla 11. Operaciones de la API

Id	Operación	Propósito	Éxito
E-01	POST /api/v1/quotes	Cotiza el carrito y sella la cotización	201 con Location
E-02	GET /api/v1/quotes/{quoteId}	Lee la cotización y su estado	200

		derivado	
E-03	POST /api/v1/orders	Confirma una cotización; exige Idempotency-Key	201 con Location
E-04	GET /api/v1/orders/{orderId}	Lee un pedido, presentado desde su cotización	200
E-05	GET /api/v1/orders?customerId=	Los pedidos del cliente, del más reciente al más antiguo, 20 por página por defecto y 100 como máximo	200
E-06	GET /actuator/health	Salud, sin cabeceras de tenant	200
E-07	GET /api/v1/products	Los productos del tenant en orden de SKU, solo lectura, sin datos de palancas, clientes ni cupo	200

Los errores usan el formato de detalles de problema de RFC 9457 (Nottingham et al., 2023): application/problem+json con type, un title y un detail localizados, status, instance, un code estable y un correlationId que también aparece en cada línea de log de la petición. Los códigos de estado conservan su significado HTTP (Fielding et al., 2022), como muestra la Tabla 12. Solo ProblemDetailsAdvice traduce los errores tipados del dominio y de la aplicación a estados, y ningún cuerpo expone un stack trace ni SQL.

Tabla 12. Estados y códigos de error

Estado	Significado	Códigos
400	Petición mal formada	VALIDATION_FAILED, INVALID_QUANTITY, CART_EMPTY, TENANT_NOT_FOUND, IDEMPOTENCY_KEY_MISSING, IDEMPOTENCY_KEY_INVALID
404	No existe, o no es visible para quien llama	QUOTE_NOT_FOUND, ORDER_NOT_FOUND, CUSTOMER_NOT_FOUND
404, 405, 406, 415	Ruta, método o tipo de contenido rechazado por el framework	REQUEST_NOT_SUPPORTED
409	Conflicto con el estado actual	QUOTE_EXPIRED, QUOTE_ALREADY_CONFIRMED, CREDIT_LIMIT_EXCEEDED, IDEMPOTENCY_KEY_IN_PROGRESS
422	Petición válida que rompe una regla de negocio	UNKNOWN_SKU, CREDIT_NOT_ENABLED, IDEMPOTENCY_KEY_REUSED
500	Inesperado, registrado con el id de correlación	INTERNAL_ERROR

Una cotización, un pedido o un historial ajeno responde 404, no 403, para no revelar nunca que existe el recurso de otro cliente (decisión de comportamiento BD-01).

Cálculo del precio

Pipeline de cálculo y orden de las palancas

El motor primero normaliza el carrito: une los SKU repetidos, ordena las líneas por código de SKU y exige cantidades enteras de 1 a 1,000,000 (regla BR-00), así que el resultado nunca depende del orden en que el cliente envió las líneas. Como muestra la Figura 5, luego aplica las palancas en un orden fijo, y solo dos pasos redondean.

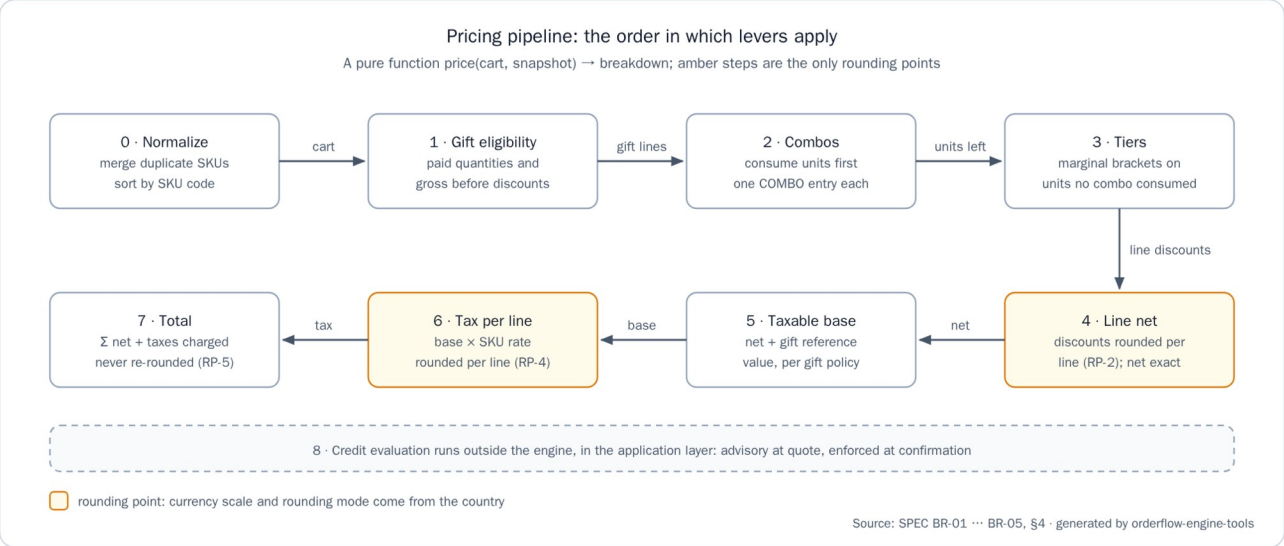


Figura 5. Pipeline de cálculo: el orden en que se aplican las palancas

La Tabla 13 da el orden y la razón de cada posición, como los fijan SPEC BR-01 y el README de la API.

Tabla 13. Orden de las palancas y su justificación

Paso	Palanca	Por qué en esta posición
1	Elegibilidad de obsequios, sobre cantidades pagadas o el subtotal bruto antes de descuentos	El obsequio lo activa lo que se compra y nunca reduce lo que se paga por producto
2	Combos	El precio negociado del combo consume primero sus unidades, para que no reciban además descuento por escala
3	Descuento por escala, por tramos marginales, sobre las unidades que no consumió un combo	El incentivo genérico por volumen aplica solo a lo que el combo no tarifó
4	Neto de la línea = bruto − descuentos	Los descuentos comerciales se liquidan antes del impuesto
5	Base gravable: el neto, más el valor de referencia del obsequio donde el país lo grava	El enunciado exige el impuesto sobre la base después de los descuentos comerciales
6	Impuesto por línea, a la tasa de cada SKU	Las tasas son por SKU
7	Total = Σ neto + impuestos	Necesita todos los pasos anteriores
8	Verificación de cupo, en la capa de	Necesita el total final; es informativa al

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 14

	aplicación	cotizar y obligatoria al confirmar

Tres lecturas completan el orden. El enunciado dice que las escalas son "acumulativas por tramos"; la suposición A-01 lo lee como tramos marginales, donde cada porcentaje aplica solo a las unidades dentro de su tramo, y el descuento de la línea se redondea una vez por línea, nunca por segmento de tramo. Una regla por SKU prevalece sobre una regla por familia para ese SKU (A-02). Un combo aparece como exactamente una entrada de descuento COMBO:<id>, sus productos nunca muestran descuentos TIER sueltos, y cada línea participante lleva comboAllocations, que solo sirven para derivar su base gravable (A-07); un combo que no bajaría el precio no consume nada. Un obsequio es una línea aparte con su cantidad y su origen GIFT:<id>, con precio cero, y la política del país para obsequios decide si su valor de referencia se grava.

El golden test 1 muestra la lectura marginal sobre la semilla de Colombia. Veinticinco colas a 2,500 dan un bruto de 62,500. Las 10 unidades del tramo 10–19 reciben el 5% (1,250) y las 6 unidades desde la 20 reciben el 10% (1,500), un descuento de 2,750. El neto de 59,750 es la base gravable, el impuesto del 19%, 11,352.50, se redondea hacia arriba en la mitad a 11,353, y el total es 71,103.

Dinero y redondeo

El dinero nunca es un número de punto flotante binario (P-01), según el viejo consejo de evitar float y double cuando se necesitan respuestas exactas (Bloch, 2018). Money es un record inmutable que contiene un BigDecimal (Oracle, s. f.) y una CurrencyPolicy. La política lleva el código ISO de la moneda, la escala y el modo de redondeo, leídos de las tablas currencies y countries y nunca de java.util.Currency, que reporta el COP con escala 2 (trampa TR-01). Un monto con más decimales que su escala se rechaza, los montos se comparan con compareTo, y la única llamada que redondea es CurrencyPolicy.round. En la base de datos, el dinero es numeric(19,4) y las tasas son numeric(7,6); en la API, el dinero es un string.

Se redondea solo por línea. La Tabla 14 lista los cinco puntos de redondeo de SPEC §4; solo RP-2 y RP-4 redondean.

Tabla 14. Puntos de redondeo

Punto	Dónde	Regla
RP-1	Precio unitario × cantidad	Exacto, porque los precios ya están en la escala de la moneda
RP-2	Cada componente de descuento de una línea (escala, asignación de combo)	Se redondea a la escala de la moneda; un combo se reparte por residuo mayor para que sus partes sumen exacto; un componente que redondea a cero no se emite
RP-3	Neto = bruto − componentes	Exacto
RP-4	Impuesto por línea	Se redondea a la escala de la moneda
RP-5	Totales del pedido	Sumas de valores de línea ya redondeados, nunca se vuelven a redondear

Se eligió redondear por línea porque una factura electrónica lleva el impuesto por línea, así que el pedido siempre coincide con la factura que produce. Se descartó redondear por grupo de tasa porque los valores de línea no cuadrarían con los totales, y se descartó

redondear solo el total porque esconde la deriva de centavos y no se puede auditar línea por línea. El golden test 4 muestra el efecto: un descuento de combo de 3,600 se reparte en 2,647 y 953, los impuestos son 1,397 al 19% y 132 al 5%, y el total es 11,529.

Cotizaciones y confirmación

Cotizaciones selladas: cotizado igual a confirmado

Como muestra la Figura 6, POST /api/v1/quotes normaliza el carrito, carga mediante SnapshotProvider un snapshot de precios, palancas activas, tasas de impuesto y política del país en el instante T, ejecuta el motor puro, agrega el resultado informativo del cupo cuando el pago es CREDIT, e inserta la cotización con sus líneas y sus componentes de descuento. Cada línea captura unit_price y tax_rate como estaban en T y guarda taxable_base y tax ya redondeados. Estas filas son de solo inserción: no existe ninguna sentencia UPDATE sobre cotizaciones (nota DM-02 del modelo de datos).

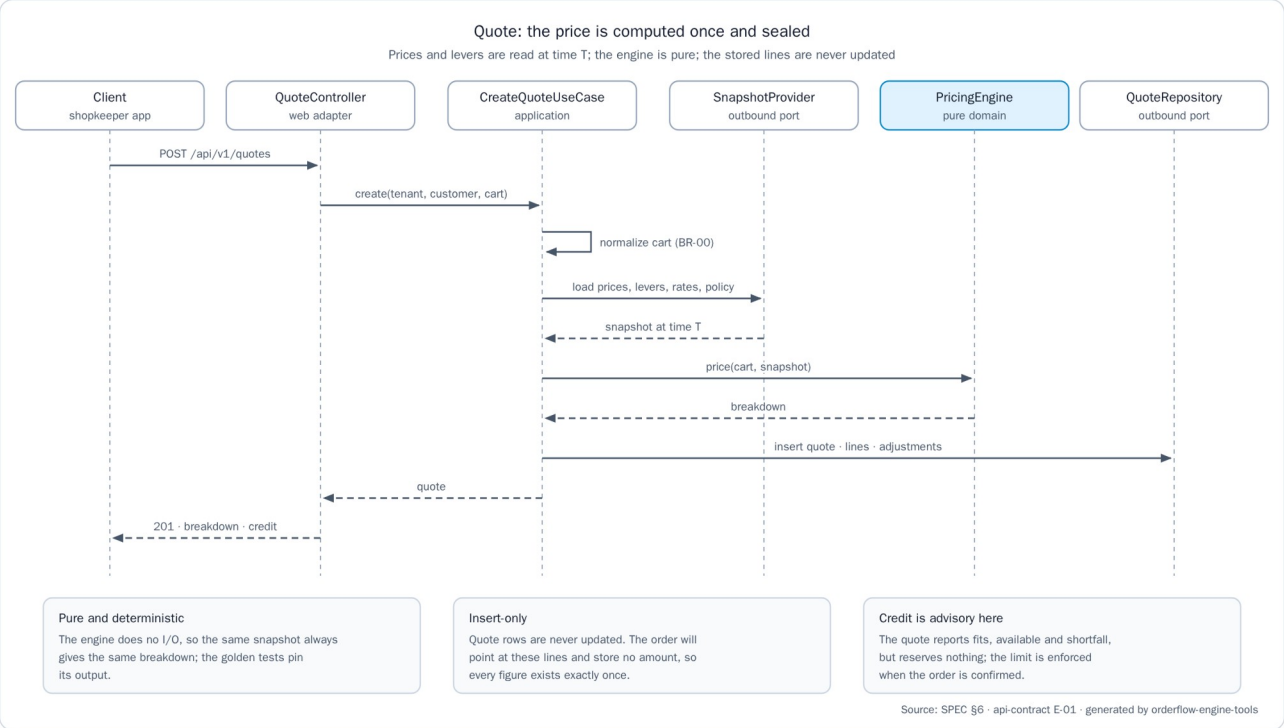


Figura 6. Cotización: el precio se calcula una vez y se sella

La confirmación no escribe montos. La fila del pedido guarda su número y una referencia a la cotización, y su desglose se presenta desde las líneas de la cotización, así que cada cifra existe una sola vez y el total confirmado es el cotizado por construcción (P-02). El motor nunca se vuelve a ejecutar al confirmar, cosa que hace cumplir una regla de ArchUnit; si no, un despliegue que cambiara el motor alteraría todas las cotizaciones abiertas. El golden test 6 prueba el mecanismo contra PostgreSQL: entre la cotización y la confirmación cambia un precio y se desactiva una palanca, el total confirmado sigue siendo igual al cotizado, y una cotización nueva da otro total, lo que demuestra que la prueba puede fallar.

El estado de la cotización se deriva y nunca se guarda, así que una fila de cotización nunca se actualiza:

```
OPEN ──confirm──▶ CONFIRMED   (existe un pedido para la cotización)
  └──ttl─────────▶ EXPIRED     (now > expiresAt, y sin pedido)
```

expiresAt es el instante de creación más el tiempo de vida de cotización del país, 15 minutos en todos los países sembrados. La concesión es explícita: un cambio de precio aplica solo a cotizaciones nuevas, así que un precio equivocado se respeta como mucho durante un tiempo de vida. Ese es el riesgo que registra el ADR.

La transacción de confirmación

ConfirmOrderUseCase ejecuta SPEC §7.1 en una sola transacción ACID en READ COMMITTED con bloqueos de fila explícitos (PostgreSQL Global Development Group, s. f.). Como muestra la Figura 7, el 201 se serializa solo después de que vuelve el commit, y nada dentro de la transacción llama a un sistema remoto. Se eligió SQL explícito con el JdbcClient de Spring en lugar de JPA porque JPA oculta los bloqueos y el orden de flush de los que dependen estas invariantes (DEC-05).

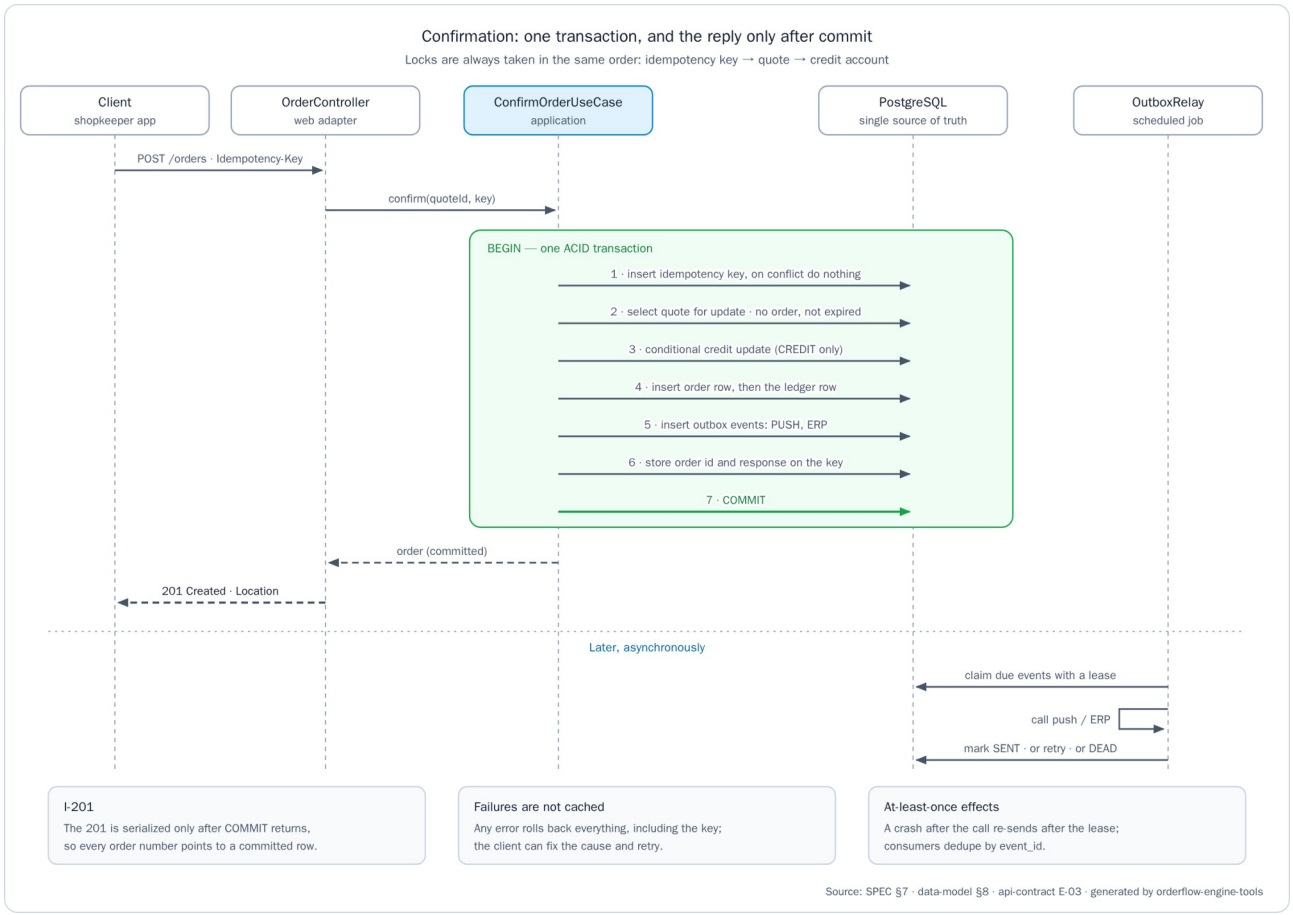


Figura 7. Confirmación: una transacción, y la respuesta solo después del commit

La Tabla 15 lista los pasos y la garantía que da cada uno.

Tabla 15. Pasos de la transacción de confirmación

Paso	Acción	Garantía
1	Insertar la clave de idempotencia con ON CONFLICT DO NOTHING y leerla de vuelta	Un solo dueño por clave; un duplicado concurrente espera sobre la entrada sin confirmar
2	Bloquear la cotización con SELECT …	Una sola confirmación por cotización

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 17

	FOR UPDATE; verificar su dueño, que no tenga pedido y que no haya vencido	
3	Si se paga a crédito, ejecutar la actualización condicional del cupo	El cupo nunca se consume por encima del límite
4	Insertar el pedido (número de la secuencia del país, confirmed_at) y luego, a crédito, la fila del libro de cupo	Ningún pedido sin su cotización; orders.quote_id es único
5	Insertar un evento de outbox por destino, PUSH y ERP	Los efectos quedan registrados de forma atómica con el pedido
6	Guardar el id del pedido y el cuerpo del 201 en la clave de idempotencia	Las repeticiones devuelven la misma respuesta
7	Commit, y luego enviar el 201	La invariante del 201

Idempotencia

La clave pertenece al tenant y al cliente y debe cumplir ^[A-Za-z0-9_-]{8,64}$. La petición se identifica con request_hash, el SHA-256 del método, la ruta, el cliente y el cuerpo JSON canónico. Guardar las claves en la transacción del pedido, y no en una caché, las vuelve atómicas con el pedido (DEC-08); este tipo de reintentos idempotentes es lo que vuelve seguras las peticiones con entrega al menos una vez (Kleppmann, 2017). La Tabla 16 lista los resultados.

Tabla 16. Reintentos y repeticiones

Situación	**Respuesta**
Misma clave, misma petición, ya completada	La respuesta guardada, idéntica byte a byte, con Idempotent-Replayed: true
Misma clave, otra petición	422 IDEMPOTENCY_KEY_REUSED
Misma clave mientras la primera petición sigue en curso	Espera y luego repite; 409 IDEMPOTENCY_KEY_IN_PROGRESS si antes vence el tiempo de espera de bloqueo
Clave ausente o mal formada	400 IDEMPOTENCY_KEY_MISSING o IDEMPOTENCY_KEY_INVALID
Una petición fallida (validación, cotización vencida, cupo excedido)	No se cachea: la transacción se revierte y libera la clave, así que el cliente puede corregir la causa y reintentar
Una clave nueva sobre una cotización ya confirmada	409 QUOTE_ALREADY_CONFIRMED, con el número del pedido existente

Las claves se guardan 24 horas. Pasado ese plazo, un reintento sigue recibiendo QUOTE_ALREADY_CONFIRMED con el número del pedido, porque los pedidos nunca se purgan.

Orden de bloqueos y libro de cupo

Los bloqueos se toman siempre en el mismo orden, clave de idempotencia, luego cotización, luego cuenta de cupo, así que las confirmaciones concurrentes no pueden entrar en deadlock. Cada espera de bloqueo está acotada por un lock_timeout local a la transacción de dos segundos (ConfirmOrderUseCase.LOCK_TIMEOUT), de modo que

una petición atascada produce un 409 que el cliente puede reintentar, en lugar de quedarse colgada.

El cupo se consume con una sola sentencia condicional: UPDATE credit_accounts SET consumed = consumed + :total … WHERE … AND consumed + :total <= credit_limit. Si no actualiza ninguna fila, la respuesta es 409 CREDIT_LIMIT_EXCEEDED y el rollback no deja pedido ni fila en el libro de cupo. Si la actualiza, se agrega una fila en credit_ledger justo después de la fila del pedido. Un CHECK (consumed >= 0 AND consumed <= credit_limit) hace que el límite se cumpla aunque el código de la aplicación estuviera mal, y consumed siempre es igual a la suma del libro, cosa que afirma CreditLimitIT. Al cotizar, el resultado del cupo (fits, available, shortfall) es informativo y no reserva nada (A-08).

La invariante del 201

La invariante I-201 dice que el cuerpo del 201 se serializa solo después del commit de la transacción que insertó la fila del pedido y tomó su número, y que el detalle del pedido y el historial leen las mismas tablas primarias en READ COMMITTED. Por lo tanto, nunca se devuelve un número de pedido sin una fila confirmada y legible. SPEC §7.6 relaciona cada causa raíz plausible del defecto reportado con una respuesta de diseño y una prueba, como muestra la Tabla 17.

Tabla 17. Cómo se vuelve imposible "confirmado pero inexistente"

Causa raíz plausible	Respuesta de diseño	Verificado por
Respuesta enviada antes del commit	La respuesta se serializa tras el commit	ConfirmOrderCommitOrderingIT
Transacción saltada por una llamada desde la misma clase	La frontera transaccional es un bean de caso de uso aparte, y ArchUnit lo hace cumplir	TransactionBoundaryArchTest, ConfirmOrderFailureInjectionIT
Historial leído de una réplica o del ERP	El historial lee las tablas primarias en la misma petición	OrderReadYourWritesIT
Pedido escrito en segundo plano después de responder	La fila del pedido se escribe en la transacción de la petición; solo los efectos se difieren	ConfirmOrderFailureInjectionIT
Actualización perdida o duplicado por doble toque	Clave de idempotencia única, quote_id único, actualización condicional del cupo	ConfirmOrderConcurrencyIT

Modelo de datos

El modelo es relacional, en tercera forma normal y sin columnas JSON, así que cada hecho es una columna que se puede restringir y consultar (DEC-19). Las tablas de cada tenant usan la clave compuesta (tenant_id, id), y toda clave foránea entre ellas incluye tenant_id, de modo que ninguna fila puede referenciar datos de otro tenant y las tablas se pueden particionar por tenant más adelante sin cambiar las claves. Los identificadores son UUIDv7 generados por la aplicación, así que las inserciones quedan locales en el

índice. Como muestra la Figura 8, las 19 tablas se agrupan en cuatro bloques, y el pedido no guarda ningún monto del desglose.

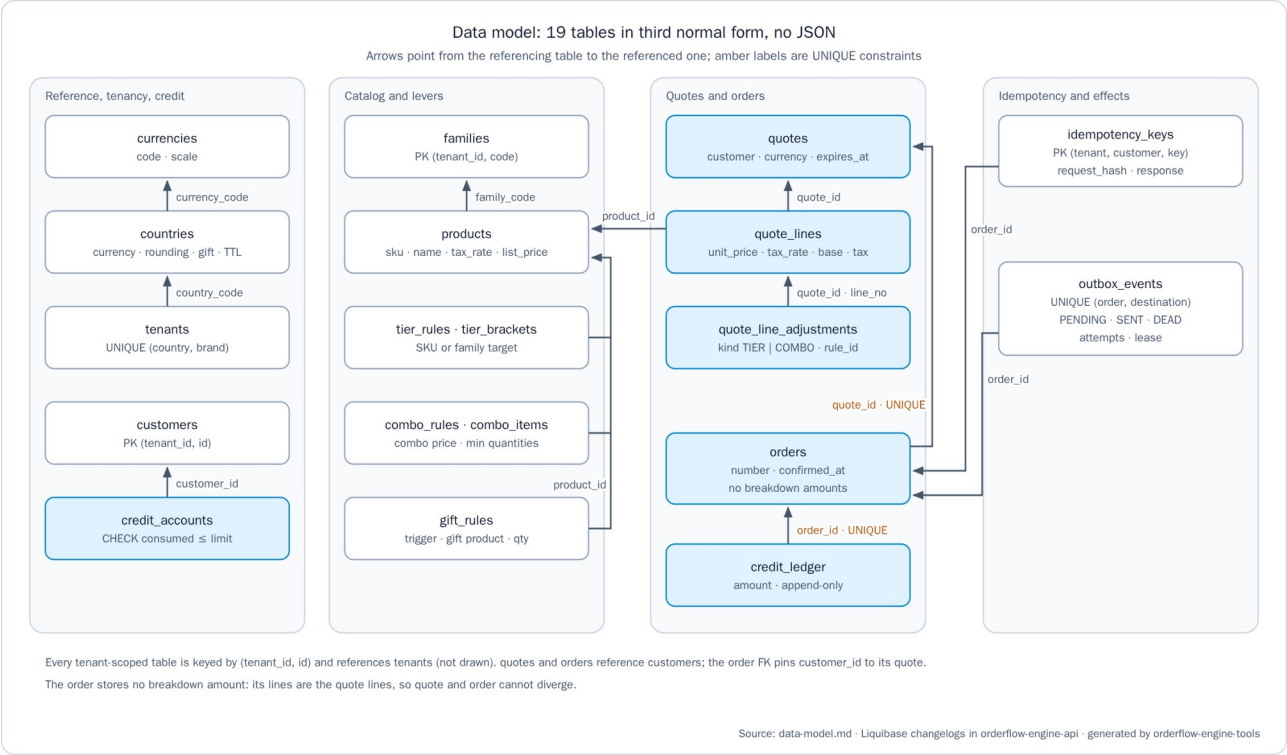


Figura 8. Modelo de datos: 19 tablas en tercera forma normal, sin JSON

La Tabla 18 lista las tablas por grupo con la invariante que sostiene cada grupo.

Tabla 18. Tablas principales del modelo de datos

Grupo	Tablas	Invariante
Referencia	currencies, countries, tenants	La escala pertenece a la moneda; el modo de redondeo, la política de obsequios y el tiempo de vida de la cotización pertenecen al país; un tenant es un par (país, marca) único
Catálogo y palancas	customers, families, products, tier_rules, tier_brackets, combo_rules, combo_items, gift_rules	Cada producto lleva su tasa de impuesto y su precio de lista; una sola regla de escala activa por destino; cada regla de obsequio tiene exactamente un disparador
Cupo	credit_accounts, credit_ledger	consumed nunca supera credit_limit; el libro es de solo agregado, una fila por pedido, y consumed es igual a su suma
Cotizaciones	quotes, quote_lines, quote_line_adjustments	Solo inserción; las líneas capturan unit_price y tax_rate

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 20

		y guardan taxable_base y tax redondeados; cada ajuste es un componente redondeado, de tipo TIER o COMBO
Pedidos	orders	Ningún monto del desglose; un pedido por cotización; un número único por tenant; el cliente queda atado a la cotización por una clave foránea compuesta
Idempotencia y efectos	idempotency_keys, outbox_events	Un dueño por clave, con el hash SHA-256 en bruto y la respuesta guardada; un evento por destino por pedido, en estado PENDING, SENT o DEAD

Las pocas columnas que parecen derivadas son deliberadas. Los hechos capturados (unit_price, tax_rate, currency_code, expires_at) registran lo vigente en el instante T. Los resultados guardados (taxable_base, tax) registran lo que produjeron la política del país y el redondeo, porque recalcularlos después no es seguro. consumed es una suma materializada que protege su CHECK, y response_body existe porque una repetición debe ser idéntica byte a byte.

El esquema lo gestiona Liquibase (Liquibase, s. f.). Los changelogs viven en db/changelog/YYYY/YYYY-MM-DD-NNN-verb-object.yaml, cada changeset tiene un rollback que prueba LiquibaseRollbackIT, y cada archivo fija un logicalFilePath, así que los archivos se pueden mover sin volver a aplicarse. Los changelogs viven con el servicio, su único dueño, y se aplican cuando arranca (DEC-20).

El relay del outbox y su ciclo de vida

Todo efecto externo pasa por el outbox, escrito en la misma transacción que el pedido (P-04). Se descartó publicar después del commit porque una caída perdería el evento, y se descartó el commit en dos fases porque push y el ERP no pueden participar en él (DEC-09). Este es el patrón de outbox transaccional (Richardson, s. f.).

Como muestra la Figura 9, un evento se escribe como PENDING. Cada consulta de OutboxRelay reclama hasta 20 eventos pendientes con FOR UPDATE SKIP LOCKED en una transacción corta que hace commit de inmediato, y fija un lease (locked_until), así que pueden correr varias instancias del relay a la vez. Luego el relay llama a PushNotificationPort o a ErpPublisherPort fuera de toda transacción, y pasa el event_id como clave de idempotencia aguas abajo. Un éxito marca el evento SENT. Un fallo reintentable incrementa attempts y programa next_attempt_at con backoff exponencial y jitter completo; tras el último intento, o de inmediato ante un error que reintentar no corrige, como un 4xx del ERP, el evento pasa a DEAD, una línea de log ERROR pide un operador, y el gauge outbox_dead lo cuenta. Solo quien tiene el lease marca el resultado.

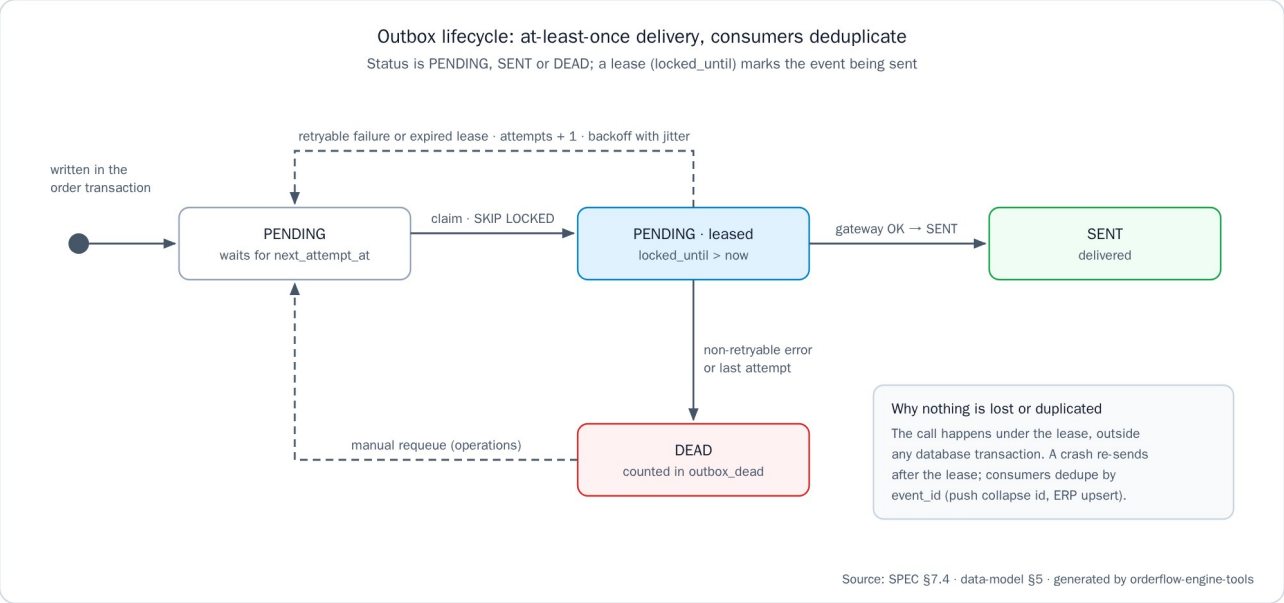


Figura 9. Ciclo de vida del outbox: entrega al menos una vez, y los consumidores deduplican

La entrega es al menos una vez. Si el proceso se detiene entre el envío y la marca, el lease vence y el evento se envía de nuevo, y los consumidores deduplican por event_id: push lo usa como collapse id y el ERP hace upsert por número de pedido, así que el efecto ocurre una sola vez (Kleppmann, 2017). Cuando un sistema externo está caído, el pedido sigue confirmado, el efecto espera en el outbox, el gauge outbox_pending muestra el atraso, y la entrega se reanuda cuando el sistema se recupera. El README de la API documenta la sentencia segura y repetible que reencola un evento DEAD después de corregir su causa.

Un job de limpieza corre cada hora para todos los tenants. Purga las claves de idempotencia a las 24 horas, las cotizaciones sin confirmar 7 días después de vencer, y los eventos SENT 7 días después de escritos; los pedidos, sus cotizaciones y los eventos DEAD nunca se borran. La Tabla 19 lista las claves de configuración y sus valores por defecto, que .env.example refleja con un comentario por clave.

Tabla 19. Configuración del outbox, la retención y el idioma

Clave	Por defecto	Significado
app.outbox.poll-interval	1 s	Espera entre dos consultas del relay
app.outbox.lease	30 s	Cuánto dura el lease de un evento reclamado
app.outbox.max-attempts	8	Intentos antes de que un evento pase a DEAD
app.outbox.backoff.base, app.outbox.backoff.max	2 s, 5 min	Límites del backoff exponencial
app.idempotency.retention	24 h	Cuánto se guardan las claves de idempotencia
app.housekeeping.retention	7 días	Retención de las cotizaciones vencidas y de los eventos SENT
app.i18n.default-locale	es	Idioma cuando Accept-Language

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 22

		falta o no está soportado

Multi-tenant y multipaís

Un tenant es un par (país, marca). La cabecera X-Tenant-Id resuelve el tenant, el tenant resuelve la política de su país, y un tenant desconocido responde 400 TENANT_NOT_FOUND. Precios, palancas, tasas de impuesto, clientes y cuentas de cupo son datos de cada tenant, y ninguna moneda, escala, tasa ni regla está escrita en la lógica (P-07). La Tabla 20 lista los cuatro tenants sembrados, uno por cada país que nombra el enunciado.

Tabla 20. Tenants sembrados

Tenant	País y política de dinero	Impuestos y palancas sembrados	Tiendas del selector web
co-alpha	Colombia, COP, escala 0, HALF_UP, obsequios TAXED	19% y 5%; una regla de escala por SKU, un combo y un obsequio repetible	Tienda Norte, Tienda Sur
ec-alpha	Ecuador, USD, escala 2, HALF_UP, obsequios EXEMPT	15%; una regla de escala por familia, un combo y un obsequio por subtotal bruto	Tienda Quito, Tienda Guayaquil
gt-alpha	Guatemala, GTQ, escala 2, HALF_UP, obsequios EXEMPT	12%; una regla de escala por SKU y un obsequio repetible	Tienda Ciudad de Guatemala, Tienda Quetzaltenango
pe-alpha	Perú, PEN, escala 2, HALF_UP, obsequios TAXED	18%; un combo y un obsequio por subtotal bruto	Tienda Lima, Tienda Arequipa

> **Nota:** Los datos semilla son de demostración, elegidos para que las pruebas cubran las dos escalas y las dos políticas de obsequios. No son una declaración de la ley tributaria real.

Cada país tiene su propio cliente con cupo (cupos de 500,000 COP, 1,500.00 USD, 5,000.00 GTQ y 2,000.00 PEN), y el tiempo de vida de la cotización es de 15 minutos en todos. Los números de pedido salen de una secuencia por país, order_number_seq_co, _ec, _gt y _pe, con el formato CO-000042. Los números se toman dentro de la transacción del pedido, así que un rollback deja un hueco; los huecos son aceptables y los duplicados no. Guatemala y Perú se agregaron en la tarea T-20 solo como datos, con el changelog 2026-10-16-002-seed-gt-and-pe.yaml y sus dos secuencias, sin cambiar el motor ni el código de la API, lo que por sí solo prueba que nada está fijo en el código. Un país nuevo necesita por tanto sus filas y su secuencia; la administración del catálogo sigue fuera del alcance, así que las tiendas, los productos y las palancas nuevos entran por changesets de Liquibase.

Internacionalización

El texto para el usuario final nunca vive en el código (P-09). El español es el idioma por defecto y el inglés está completo, y agregar un idioma significa agregar un archivo de cada lado. La Tabla 21 resume los dos mecanismos.

Tabla 21. Mecanismos de internacionalización

Lado	Mecanismo y archivos	Guarda
API	MessageSource de Spring en UTF-8, nunca el locale de la JVM: messages.properties (base en español), messages_es.properties, messages_en.properties	ErrorCatalogI18nTest falla cuando a un código de error le falta su mensaje en algún bundle
Web	ngx-translate con diccionarios JSON en tiempo de ejecución: es.json y en.json en public/i18n	i18n-keys.spec.ts, no-literal-text.mjs, y la regla de paridad de claves de check-docs

Del lado de la API, las capas de dominio y de aplicación llevan solo códigos de error y argumentos, y el adaptador web resuelve el title y el detail localizados según Accept-Language, con es como respaldo; los logs quedan en inglés. Del lado web, el selector ES/EN o ?lang=en cambia el idioma en tiempo de ejecución, la elección se guarda en el almacenamiento del navegador, y cada llamada a la API envía el idioma activo en Accept-Language, así que los textos de los problemas llegan en el idioma de la pantalla. Cada diccionario nombra en app.formatLocale el locale que da formato al dinero, los números y las fechas: es-419 para español y en para inglés. Se descartó la internacionalización integrada de Angular porque genera un bundle por locale y no puede cambiar de idioma en ejecución (DEC-11). Los nombres de productos y tiendas son datos del tenant en su idioma, y el cliente nunca los traduce.

Arquitectura del cliente web

Pantallas y rutas

El cliente es una capa delgada sobre el contrato, construida con componentes standalone de Angular (Google, s. f.). Nunca calcula dinero, nunca guarda un monto y muestra las cifras y los textos de problema de la API tal como llegan. La Tabla 22 lista sus pantallas.

Tabla 22. Pantallas del cliente web

Ruta	Pantalla	Llamadas
/cart, también /	Carrito: grid de productos con imagen, nombre, SKU, precio de lista antes de impuestos, tasa de impuesto y selector de cantidad; contado o crédito	E-07, luego E-01
/quotes/:quoteId	Resumen del pedido, la pantalla del enunciado: estado, vencimiento, el desglose completo y la posición informativa del cupo	E-02 en cada visita, luego E-03
/orders/:orderId	Pedido: número, hora de confirmación, medio de pago y el mismo desglose	E-04
/orders	Mis pedidos: número, fecha y total, del	E-05 en cada

	más reciente al más antiguo	visita

Un selector de tienda lista las ocho tiendas sembradas de public/config/stores.json y hace las veces de autenticación (A-06). Un solo interceptor HTTP pone X-Tenant-Id, X-Customer-Id y Accept-Language en cada llamada a /api/, y las pantallas nunca ponen cabeceras. El carrito vive solo en el navegador, uno por tienda, y se vacía tras un 201.

Un solo origen a través de nginx

El cliente llama solo rutas relativas /api/v1/…, y quien lo sirve reenvía /api/ a la API, así que el navegador nunca hace una llamada entre orígenes y la API no tiene configuración de CORS. La imagen de nginx escribe API_UPSTREAM en su configuración cuando arranca el contenedor, de modo que una sola imagen corre contra cualquier API y una dirección nueva necesita un reinicio, no una recompilación. proxy_pass no lleva URI, así que las rutas y cabeceras como Idempotency-Key, Location e Idempotent-Replayed pasan sin cambios (nginx, s. f.); cualquier otra ruta cae en index.html, así que recargar una ruta del cliente funciona; y /actuator no se reenvía. La Tabla 23 lista las tres formas de servir el cliente: con nginx en el contenedor web en las dos primeras, y con el servidor de desarrollo de Angular en la tercera.

Tabla 23. Formas de servir el cliente

Modo	Reenvía `/api/` a	Lo fija
Stack completo	api:8080, el servicio de la API en la red de Compose	API_UPSTREAM en el Compose de infraestructura
Cliente web solo	host.docker.internal:8080, una API en el host	API_UPSTREAM en el .env.example del cliente web
Servidor de desarrollo	localhost:8080	proxy.conf.mjs

Prevención a nivel de componente

El enunciado pregunta cómo evitar los tres defectos pasados a nivel de componente y no de pantalla. Cada defecto tiene un componente que es su dueño, y las pantallas nunca imprimen texto, ponen una acción ni dibujan una imagen de producto de otra manera, así que una pantalla nueva hereda la prevención. La Tabla 24 lista las reglas.

Tabla 24. Prevención de los tres defectos a nivel de componente

Defecto	Componente	Regla
Texto cortado	ui-text	Siempre hace salto de línea, nunca usa puntos suspensivos ni recorte de líneas, y crece en vertical; un monto va en una celda que nunca se encoge, así que un nombre largo se parte y el monto nunca se divide
Controles tapados por la barra del sistema o el teclado	ui-action-bar	Fija al fondo, con el relleno de las áreas seguras, y elevada sobre el teclado siguiendo visualViewport; cada control mide al menos 44 × 44 px

Imágenes del grid con escalas distintas	ui-product-image	Una sola relación de aspecto con object-fit: contain; una imagen que falta se reemplaza por un placeholder

Las specs miden el layout real en Chromium headless a 768 × 1024 y 375 × 812, porque un DOM simulado sin motor de layout reporta cada ancho como cero (trampa TR-11).

Dinero, idempotencia y errores en el cliente

Cada monto en pantalla es un string de un cuerpo de la API, formateado tal cual por un solo pipe money con Intl.NumberFormat, así que ningún monto se vuelve un número flotante binario. Como el COP y el USD comparten el símbolo $, cada pantalla con montos nombra el código ISO de la moneda. En el primer toque de "Confirmar pedido", el cliente crea una Idempotency-Key UUID por cotización, la guarda en el almacenamiento del navegador, deshabilita el botón mientras la petición está en vuelo, y envía la misma clave en cada reintento, también después de recargar; un 201 repetido muestra "Este pedido ya estaba registrado". Una cotización a crédito abierta que no cabe en el cupo deshabilita la confirmación y ofrece "Pagar de contado". Cada problema muestra el title y el detail de la API, su código y los primeros 8 caracteres de su correlationId, con exactamente una salida; una petición sin respuesta muestra en su lugar el texto propio del cliente para ese caso. Las garantías del servidor nunca dependen del cliente; el cliente solo evita pedir dos veces.

Infraestructura y entrega

Contenedores y Compose

Cada repositorio ejecutable se construye y corre con sus propios archivos de Docker y su propio .env.example, y un .env nunca se versiona. La imagen del servicio se compila con el wrapper en JDK 21 y corre en un JRE 21 con un usuario sin privilegios. La imagen web tiene una etapa de pruebas sobre la imagen de Playwright, una etapa de compilación en Node y una etapa de nginx sin privilegios. El Compose de infraestructura (Docker, s. f.) arranca primero PostgreSQL, la API cuando la base de datos está sana, y el cliente web después de la API, porque nginx resuelve el nombre de su upstream al arrancar (trampa TR-12). Todos los servicios tienen restart: unless-stopped, así que Docker reinicia un servicio cuando termina y el stack cuando arranca el daemon, mientras que un servicio detenido a mano queda detenido. Los valores de .env.example también son los valores por defecto del Compose, así que el stack arranca sin .env.

El comando de pruebas del espacio de trabajo

El comando único del enunciado es ./mvnw verify en orderflow-engine-api; necesita Docker en marcha porque Testcontainers levanta un PostgreSQL real (Testcontainers, s. f.). El espacio de trabajo agrega un comando que ejecuta todas las verificaciones de todos los repositorios:

```
bash orderflow-engine-infra/scripts/test-all.sh
```

Ejecuta, en orden y deteniéndose en la primera falla: la compilación y las pruebas de la API; docker build --target test . en orderflow-engine-web, así que el host no necesita Node para las specs; el flujo de punta a punta; check-docs; y check-integrity. El paso de punta a

punta verifica que la suite use la versión de Playwright que fija el cliente web, levanta el stack completo en su propio proyecto de Compose en los puertos 18080 y 18081, ejecuta la suite de Playwright (Microsoft, s. f.) en un contenedor dentro de la red del stack, y siempre elimina el stack con su volumen, así que cada corrida parte de las semillas. La suite recorre carrito, cotización, confirmación, pedido y Mis pedidos una vez en español y otra en inglés, y verifica que el pedido nuevo aparezca primero en Mis pedidos y en el historial leído de vuelta desde la API.

Modelo de ramas, fases y commits

Los repositorios de código mantienen main y dev; el repositorio de documentación mantiene solo main, porque contiene documentación viva. Cada tarea entra por uno o más pull requests desde una rama llamada <type>/T-NN-slug, que se integran con squash en dev (en main para la documentación) y luego se borran. El main de un repositorio de código recibe solo dev, cuando el dueño cierra una fase, mediante un merge commit titulado chore: close <phase> phase (T-NN). Cada commit es una línea de 60 caracteres como máximo en el formato Conventional Commits 1.0.0 con su etiqueta de tarea (Conventional Commits, s. f.), por ejemplo feat(db): seed Guatemala and Peru tenants (T-20). El enunciado no pide integración continua, así que la verificación corre en la máquina de desarrollo y su salida se pega en cada pull request. GitHub rechaza los rulesets en repositorios privados de un plan gratuito, así que las reglas de protección se siguen a mano y los rulesets se aplican cuando los repositorios se vuelvan públicos en la entrega. La Tabla 25 lista las cuatro fases.

Tabla 25. Fases del trabajo

Fase	Tareas	Cerrada el	Pull requests
1. Preparación	T-00	2026-10-14	21 de tarea y 5 de cierre de fase
2. Andamiaje	T-01	2026-10-15	9 de tarea y 2 de cierre de fase
3. Núcleo	T-02 … T-16, T-18	2026-10-15	20 de tarea y 3 de cierre de fase
4. Web	T-17, T-19, T-20	2026-10-17	18 de tarea (más el registro del handoff, docs #18, pendiente) y 4 de cierre de fase

Estrategia de calidad y evidencia

Capas de pruebas

Cada tarea sigue el mismo ciclo: escribir la prueba que falla y verla fallar por la razón esperada, implementar lo mínimo para que pase, luego romper la protección una vez para probar que la prueba muerde, y restaurarla (P-08). La Tabla 26 lista las capas de pruebas de SPEC §11.

Tabla 26. Capas de pruebas

Capa	Alcance	Herramientas
Unitaria	Money, la normalización, cada palanca, el pipeline	JUnit 5, AssertJ
Golden	Desgloses completos para entradas fijas, con reloj fijo e ids fijos	Fixtures JSON: request.json, snapshot.json, expected.json

Integración	Persistencia, transacciones, concurrencia, outbox	Testcontainers con PostgreSQL 17, Spring Boot test
Contrato	Estado HTTP, forma del problema, texto localizado	MockMvc
Arquitectura	Dominio puro, sin punto flotante, frontera transaccional	ArchUnit
Cobertura y mutación	Cobertura de líneas y ramas de pricing; detección de fallos inyectados en pricing.domain	JaCoCo (EclEmma, s. f.); PIT (Coles, s. f.)
Web	Las tres reglas de componente en cada pantalla, idempotencia del cliente, faltante de cupo, acciones ante problemas, frescura del historial, formato del dinero, claves de diccionario	Builder de pruebas unitarias de Angular con Vitest en Chromium headless (Vitest, s. f.)
Punta a punta	El flujo completo en el navegador contra el stack real, en español e inglés	Playwright, dentro de test-all

Se eligió Testcontainers en lugar de una base de datos en memoria porque un modelo de bloqueos y un dialecto SQL distintos no probarían nada sobre las invariantes de la confirmación (DEC-12). La cobertura sola no dice nada de la calidad de las aserciones, y por eso la complementan las pruebas de mutación (DEC-14).

Golden tests

Cada golden test lleva @Tag("golden"), afirma montos exactos y verifica las identidades que se cumplen en cualquier desglose, como total = Σ neto + impuestos. Los casos 1 a 5 son carpetas de fixtures en src/test/resources/golden/ (gt01-two-tier-brackets, gt02-combo-no-loose-discount, gt03-gift, gt04-combo-with-taxes y gt05-credit-exceeded), y el caso 6 corre contra PostgreSQL con los datos semilla. La Tabla 27 relaciona los seis casos que exige el enunciado con sus pruebas.

Tabla 27. Los seis golden tests del enunciado

Caso	Prueba	Aserción clave
1. Descuento en dos tramos	GoldenPricingTest	Un descuento de 2,750 y un total de 71,103
2. Combo sin descuentos sueltos	GoldenPricingTest	Una entrada COMBO y ninguna entrada TIER para las unidades consumidas
3. Obsequio con su cantidad	GoldenPricingTest	Una línea de obsequio con su cantidad y su origen GIFT
4. Combo con impuestos	GoldenPricingTest	Impuesto sobre las bases asignadas, incluido en el total de 11,529
5. Pedido a crédito que excede el cupo	GoldenQuoteTest, CreditLimitIT	fits=false con available y shortfall; confirmar responde 409

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 28

		sin pedido y sin fila en el libro de cupo
6. Cotizar y confirmar coinciden	QuoteConfirmRegressionIT	El total confirmado es igual al cotizado tras cambiar un precio y una palanca; una cotización nueva da otro total

Otros casos comparten el arnés: las dos políticas de obsequios, el redondeo en .5, Colombia frente a Ecuador, y ambos países sin palancas. QuoteLeversIT repite los casos de escala, combo y obsequio por POST /api/v1/quotes contra los datos semilla, así que las palancas quedan probadas de punta a punta y no solo en el motor.

Evidencia verificada

La Tabla 28 cita el estado verificado de HANDOFF.md §3 con el comando que produjo cada valor.

Tabla 28. Evidencia verificada

Elemento	Valor	Verificado por
Pruebas de la API	333 pruebas unitarias y 308 de integración en verde (Testcontainers con PostgreSQL 17), entre ellas los seis golden tests	test-all sobre clones nuevos, 2026-10-17
Cobertura	100% de líneas y ramas en pricing.domain (220 líneas, 58 ramas) y pricing.domain.lever (221 líneas, 85 ramas), con umbral ≥ 90%; 99.2% de las líneas y 98.0% de las ramas en todo el servicio	El reporte de JaCoCo jacoco.xml
Puntaje de mutación	185 de 185 mutaciones detectadas (100%), con umbral del 80%	./mvnw -Pmutation verify
Pruebas web	La verificación de texto literal pasa en 14 plantillas, y luego pasan 209 specs en 15 archivos en Chromium headless	La etapa de pruebas del Dockerfile web, con su caché deshabilitada
Punta a punta	Carrito, cotización, confirmación, pedido y Mis pedidos pasan en español y en inglés (2 de 2)	El paso de punta a punta de test-all
Espacio de trabajo	test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check)	test-all.sh sobre clones nuevos desde GitHub, 2026-10-17
Fases	Preparación (T-00), andamiaje (T-01), núcleo (T-02 … T-16, T-18) y web (T-17, T-19, T-20), todas cerradas en main	gh pr list --state merged en cada repositorio

> **Nota:** Con mucha carga en paralelo, Testcontainers puede agotar su tiempo de espera mientras arranca PostgreSQL, y la falla aparece como ContainerLaunchException, nunca como una aserción (trampa TR-14). Repetir la corrida en un host más tranquilo es el primer paso antes de leerla como un defecto.

Decisiones

ADR-0001

El enunciado pide un registro de decisión de una página como máximo, así que la entrega tiene exactamente uno, en el formato ligero que popularizó Nygard (2011), con la tabla de encabezado de MADR 4.0.0: orderflow-engine-api/docs/adr/0001-order-pricing-and-confirmation.md y su par en español, aceptado el 2026-10-15, con confianza alta y media en el impuesto del obsequio. check-docs limita cada archivo de idioma a 500 palabras, y las versiones en Word y PDF caben en una página A4. El registro fija cinco decisiones:

- Una cotización sellada de solo inserción que el pedido referencia sin montos, en lugar de copiar el desglose o recalcularlo al confirmar, porque las copias divergen y los precios cambian.
- BigDecimal redondeado por línea, sin volver a redondear los totales, y combos antes que escalas, en lugar de unidades mínimas enteras, un total redondeado o escalas primero, que son incómodos con tasas, descuadran la factura o descuentan dos veces.
- Una transacción ACID de PostgreSQL con bloqueos clave, cotización, cupo, y el 201 tras el commit, en lugar de un almacén en memoria, una escritura asíncrona o una saga, o claves en una caché.
- Un outbox transaccional con lease y backoff, deduplicado por event_id, en lugar de publicar tras el commit o de un commit en dos fases.
- Un monolito hexagonal con SQL explícito y repositorios separados, en lugar de microservicios, JPA o un monorepo.

Su lista para el code review pide a quien revisa buscar: nada de float ni double, dinero comparado con compareTo, ningún UPDATE sobre cotizaciones; @Transactional solo en casos de uso, sin autoinvocación ni llamadas remotas; bloqueos en el orden clave, cotización, cupo, el 201 tras el commit y ninguna falla cacheada; un relay que envía fuera de toda transacción, donde solo quien tiene el lease marca el resultado; y cada palanca con un golden test visto fallar, con los textos de usuario solo en i18n.

Decisiones técnicas del plan

El plan técnico (spec/plan.md §1) registra cada decisión técnica con sus alternativas descartadas. La Tabla 29 las resume.

Tabla 29. Decisiones técnicas y alternativas descartadas

Código	Decisión	Principal alternativa descartada, y por qué
DEC-01	Java 21 con Spring Boot, stack servlet	WebFlux: los drivers reactivos complican la única transacción bloqueante, sin necesidad de throughput (Spring, s. f.)
DEC-02	Maven Wrapper	Gradle: sin ventaja para un solo módulo; el wrapper no necesita instalación local
DEC-03	Monolito modular hexagonal con dominio de precios puro	Microservicios: una sola transacción debe abarcar pedido, cupo, clave y outbox
DEC-04	PostgreSQL con changelogs de Liquibase y rollbacks	Almacén en memoria: no puede probar atomicidad ni bloqueos de fila

Fuente: orderflow-engine-docs/delivery/architecture-report.es.md · pág. 30

DEC-05	JdbcClient con SQL explícito	JPA: oculta los bloqueos y el orden de flush
DEC-06	BigDecimal con escala por país, redondeado por línea	Unidades mínimas enteras: incómodas con tasas de impuesto
DEC-07	Cotización de solo inserción; el pedido no guarda montos	Copiar el desglose, recalcular al confirmar o bloquear precios
DEC-08	Claves de idempotencia en la transacción del pedido	Una caché: no es atómica con el pedido
DEC-09	Outbox transaccional con un relay de consulta con lease	Publicar tras el commit, commit en dos fases, captura de cambios
DEC-10	Detalles de problema de RFC 9457 con un código estable	Un sobre de error propio que reinventa un estándar
DEC-11	MessageSource y JSON de ngx-translate en ejecución	i18n integrada de Angular: una compilación por locale
DEC-12	Testcontainers con PostgreSQL	Una base de datos en memoria con otros bloqueos y otro dialecto
DEC-13	Golden tests como fixtures JSON	Aserciones en línea: difíciles de revisar, fáciles de desviar
DEC-14	JUnit 5, JaCoCo, PIT y ArchUnit	Solo cobertura; un servidor de calidad que el evaluador no puede correr
DEC-15	Repositorios hermanos con un repositorio de documentación	Un monorepo que ata componentes independientes
DEC-16	Componentes standalone de Angular	Flutter: no está instalado en el entorno de compilación
DEC-18	Gobierno de la documentación con check-docs en un repositorio de herramientas	Solo revisión manual: las reglas sin verificación se degradan
DEC-19	Tercera forma normal, sin JSON, claves compuestas por tenant, UUIDv7	Documentos JSON para cotizaciones; solo claves sustitutas globales
DEC-20	Changelogs en el servicio, aplicados al arrancar	Un job de migración: una segunda credencial y otro bloqueo de la cotización. Un repositorio de esquema: un clon del servicio quedaría sin esquema

La desviación de la especificación

La única desviación registrada es el alcance del front, que el dueño del producto amplió de una pantalla de resumen del pedido al flujo completo. SPEC §1 la contiene: el front empezó después de la fase núcleo, no agrega ninguna regla de negocio, nunca calcula dinero, agrega una sola operación de solo lectura (E-07) que sale con el carrito y se recortaría con él, y registra su tiempo medido. SPEC §14.3 lista cada elemento que va más allá del enunciado con lo que aporta, etiquetado como calidad, estabilidad o escalabilidad: por ejemplo la normalización del carrito, los códigos de error estables, las reglas de arquitectura, el umbral de cobertura, las pruebas de mutación, la limpieza por retención, los gauges del outbox, las claves encabezadas por el tenant, las verificaciones

de documentación e integridad, los archivos de Docker por repositorio, la idempotencia del lado del cliente y la operación de catálogo de solo lectura.

Lo que no se implementó a conciencia

El ADR y el README de la API listan lo que se dejó fuera a propósito. La Tabla 30 lo resume.

Tabla 30. Elementos no implementados, a propósito

Elemento	Por qué
Autenticación	Fuera del alcance; tenant y cliente llegan en cabeceras, y un gateway los inyectaría
Cobro, cancelación, liberación de cupo	No se piden
Carrito en el servidor	No se pide; el carrito viaja en la petición de cotización
Revocación de cotizaciones	No se pide; es el siguiente paso de la mitigación del riesgo del ADR
Administración de catálogo y palancas	No se pide; las migraciones siembran los datos, y E-07 solo lee
Categorías de impuesto e historial de precios	No se piden; cada SKU lleva su tasa, y cada línea de cotización conserva el precio y la tasa con que se cotizó
Clientes reales de push y ERP	Los fakes detrás de puertos prueban las garantías de entrega
Archivo OpenAPI, despliegue, CORS	No se piden; el contrato es spec/api-contract.md, y un solo origen elimina CORS
SonarQube, pipeline de CI, mutación en CI	Cada uno solo necesitaría configuración de CI más adelante; el reporte de JaCoCo y los comandos ya existen
Job de migración, roles de mínimo privilegio, repositorio de esquema	Liquibase corre al arrancar con la credencial del dueño; el plan describe cómo se adoptaría cada uno

Las rutas de adopción de spec/plan.md §3 dan, para cada uno de estos pasos futuros, el estado de hoy, cómo se adoptaría, los cambios que requiere en el proyecto y su disparador.

Riesgos y mitigaciones

El ADR registra el riesgo principal del diseño: una cotización sellada respeta un precio equivocado hasta que vence. La mitigación es un tiempo de vida de 15 minutos por país y luego una operación de revocación (de OPEN a REVOKED, respondida con 409), que se revisaría si un incidente afecta a más de 100 cotizaciones o si el tiempo de vida supera una hora. La Tabla 31 lista el registro de riesgos de spec/plan.md §4.

Tabla 31. Registro de riesgos

Código	Riesgo	Mitigación
RISK-01	Contención sobre la fila de	Una transacción corta, un lock_timeout y

	cupo de un cliente ante ráfagas de confirmaciones	un 409 explícito; medido en ConfirmOrderConcurrencyIT
RISK-02	El outbox crece mientras el ERP está caído mucho tiempo	Backoff, DEAD tras el último intento, el gauge outbox_pending y el procedimiento para reencolar
RISK-03	Un precio equivocado respetado durante el tiempo de vida de la cotización	Aceptado como el riesgo del ADR, acotado por un tiempo de vida de 15 minutos, con la revocación como siguiente paso
RISK-04	Un cambio en el motor altera en silencio los precios de las cotizaciones nuevas	Los golden tests rompen la compilación
RISK-05	Un consumidor aguas abajo no deduplica por event_id	El contrato de event_id, el collapse id de push y el upsert del ERP por número de pedido
RISK-06	Los repositorios hermanos se desalinean al separarse	El contrato vive solo en el repositorio de documentación, y check-docs verifica los enlaces
RISK-07	La ventana de cinco días	Un plan por días y una línea de corte después de T-16, con los recortes registrados en "No incluido"

El modelo de consistencia dice qué se sacrifica. Pedido, cupo, clave de idempotencia y outbox tienen consistencia fuerte en una sola base de datos, lo que sacrifica la disponibilidad mientras la base de datos está caída y serializa las confirmaciones de un mismo cliente sobre su fila de cupo. Push y el ERP tienen consistencia eventual, con entrega al menos una vez deduplicada por los consumidores, y van detrás de la confirmación por el intervalo de consulta más los reintentos.

Conclusión

Orderflow Engine cumple el corazón del enunciado por su estructura y no solo por disciplina. El total cotizado es el confirmado porque el pedido no tiene montos propios. El dinero es exacto porque un tipo de punto flotante en el dominio rompe la compilación, y el redondeo ocurre solo en dos puntos nombrados por línea. Un 201 significa un pedido confirmado y legible porque la respuesta se serializa tras el commit de una sola transacción que contiene la clave, el cupo, el pedido y sus eventos de outbox, con bloqueos en un orden fijo. Los efectos externos no se pierden ni se duplican porque salen por un outbox con lease y llevan un id sobre el que deduplican los consumidores. El comportamiento por país son datos, como demuestra la incorporación de Guatemala y Perú solo con datos, y el cliente web previene los tres defectos de interfaz reportados en componentes y no en pantallas. El diseño se verifica con pruebas golden, de integración, de arquitectura, de mutación, de componentes y de punta a punta, cuyos resultados se registran con los comandos que los produjeron, y su principal riesgo pendiente, un precio equivocado respetado durante la vida de una cotización, está acotado y tiene un siguiente paso definido.

Referencias

American Psychological Association. (2020). Publication manual of the American Psychological Association (7.ª ed.). https://doi.org/10.1037/0000165-000

Bloch, J. (2018). Effective Java (3.ª ed.). Addison-Wesley.

Cockburn, A. (2005). Hexagonal architecture. https://alistair.cockburn.us/hexagonal-architecture/

Coles, H. (s. f.). PIT mutation testing. https://pitest.org/

Conventional Commits. (s. f.). Conventional Commits 1.0.0. https://www.conventionalcommits.org/en/v1.0.0/

Docker. (s. f.). Docker Compose. https://docs.docker.com/compose/

EclEmma. (s. f.). JaCoCo Java code coverage library. https://www.eclemma.org/jacoco/

Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.

Fielding, R. T., Nottingham, M. y Reschke, J. (Eds.). (2022). HTTP semantics (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110

Google. (s. f.). Angular. https://angular.dev/

Kleppmann, M. (2017). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media.

Liquibase. (s. f.). Liquibase documentation. https://docs.liquibase.com/

Microsoft. (s. f.). Playwright. https://playwright.dev/

nginx. (s. f.). Module ngx_http_proxy_module. https://nginx.org/en/docs/http/ngx_http_proxy_module.html

Nottingham, M., Wilde, E. y Dalal, S. (2023). Problem details for HTTP APIs (RFC 9457). Internet Engineering Task Force. https://doi.org/10.17487/RFC9457

Nygard, M. (2011, 15 de noviembre). Documenting architecture decisions. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Oracle. (s. f.). BigDecimal (Java SE 21 & JDK 21). https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/math/BigDecimal.html

PostgreSQL Global Development Group. (s. f.). PostgreSQL 17 documentation. https://www.postgresql.org/docs/17/

Richardson, C. (s. f.). Pattern: Transactional outbox. Microservices.io. https://microservices.io/patterns/data/transactional-outbox.html

Spring. (s. f.). Spring Framework reference documentation. https://docs.spring.io/spring-framework/reference/

Testcontainers. (s. f.). Testcontainers for Java. https://java.testcontainers.org/

TNG Technology Consulting. (s. f.). ArchUnit user guide. https://www.archunit.org/userguide/html/000_Index.html

Vitest. (s. f.). Vitest. https://vitest.dev/